# Lecture 26 — Same mathematics, very different computational cost

**Fri Oct 30**

Last updated: October 19, 2026.

**Boyd video (optional):** Lecture 6 — complexity · ENGR108 playlist (Boyd)

**Textbook:** Boyd & Vandenberghe, *VMLS* (PDF)

Reading: Boyd's $ab^\mathsf{T}c$ example. Related objectives: Lectures. Schedule.

Same mathematical answer can mean different storage and runtime depending on order — useful later for the capstone.

Bring paper. Warm-up, fill-in notes, board examples, and **Your turn**.

## Learning objectives

By the end of class, you should be able to:

1. Compare algebraically equivalent orders of computation.
2. Count intermediate values that must be stored.
3. Measure runtime for increasing problem sizes.
4. Explain why forming a large intermediate matrix may be wasteful.
5. Choose a computational order and justify it quantitatively.

## Warm-up

Let $a = (1, 2)$, $b = (3, 4)$, $c = (1, -1)$ (column vectors).

(a) Form the outer product $ab^\mathsf{T}$ (a $2 \times 2$ matrix).

(b) Compute $(ab^\mathsf{T})c$ and $a(b^\mathsf{T}c)$. Same vector?

## Fill in

**Associativity.** For compatible sizes, $(XY)z = X(Yz)$. In particular, for $n$-vectors $a, b, c$,

$$(ab^\mathsf{T})c = a(b^\mathsf{T}c).$$

Same mathematical result — different _____.

**Order 1 — form the matrix first.** Compute $M = ab^\mathsf{T}$ ($n \times n$), then $Mc$.

- Intermediate storage: about _____ numbers (the whole matrix $M$).
- Cost idea: building $M$ and multiplying by $c$ scales like _____ work (order $n^2$).

**Order 2 — scalar first.** Compute $s = b^\mathsf{T}c$ (one number), then $sa$.

- Intermediate storage: the scalar $s$, plus the vectors — order _____ numbers, not $n^2$.
- Cost idea: two inner/scale passes — order _____ work.

**Why wasteful?** Forming $ab^\mathsf{T}$ materializes a dense $n \times n$ matrix you may never need as an object; then you immediately collapse it with $c$. Capstone theme: choose an algorithm that matches the structure, not only the formula on the page.

**Runtime.** In software you can time both orders for growing $n$ and justify which order to use with measured times (and storage counts).

## Worked in class

**W1.** Algebra. With $a = (2, 0, -1)$, $b = (1, 1, 1)$, $c = (1, 0, 1)$, compute $s = b^\mathsf{T}c$ and $a(b^\mathsf{T}c)$. Then form $ab^\mathsf{T}$ and $(ab^\mathsf{T})c$. Match?

**W2.** Storage count. For general $n$, how many entries does $ab^\mathsf{T}$ have? How many numbers do you store for the scalar-first method beyond the three input vectors?

**W3.** Choose an order. You need $y = (ab^\mathsf{T})c$ for $n = 5000$ once. Which order do you run, and what two quantitative reasons (storage + flops/runtime idea) support the choice?

**W4 (preview).** Sketch what a timing table should look like: rows $n = 200, 1000, 3000$; columns "form $ab^\mathsf{T}$ then multiply" vs "scalar first." What qualitative pattern do you expect?

## Your turn

**Y1.** Let $a = (1, -1, 0)$, $b = (2, 1, 3)$, $c = (0, 1, 1)$. Compute both $(ab^\mathsf{T})c$ and $a(b^\mathsf{T}c)$.

**Y2.** For $n = 100$, about how many intermediate entries does Order 1 store in $ab^\mathsf{T}$? Order 2 in the scalar $s$?

**Y3.** One sentence: why can two algebraically equal expressions have wildly different runtimes?

## Answers (Your turn)

Y1. $b^\mathsf{T}c = 2 \cdot 0 + 1 \cdot 1 + 3 \cdot 1 = 4$, so $a(b^\mathsf{T}c) = 4a = (4, -4, 0)$. The matrix route gives the same:

$$ab^\mathsf{T} = \begin{bmatrix} 2 & 1 & 3 \\ -2 & -1 & -3 \\ 0 & 0 & 0 \end{bmatrix}, \qquad (ab^\mathsf{T})c = (4, -4, 0).$$

Y2. Order 1 stores $n^2 = 10{,}000$ entries in $ab^\mathsf{T}$. Order 2 stores one scalar $s$ as the essential intermediate (beyond the inputs).

Y3. Associativity preserves the value but not the work: different parenthesizations create different intermediates (e.g. an $n \times n$ matrix vs a scalar), so flops and memory — and therefore runtime — can differ by orders of magnitude.